

姓名：_____陈英杰_____

学号：_____20232673_____

《GNSS 定位新技术及数据处理方法》课后大作业（一）

GNSS 精密单点定位

任务部分：在《卫星导航定位原理》单点定位程序基础上，参考 PPPH 算法或 RTKLIB 算法，实现单点定位精度数量级的提升（到分米级或厘米级）。

选做部分：实现 mm 的静态或动态精密单点定位。

一、算法

精密单点定位（PPP）是在单站条件下，利用精密轨道、精密钟差以及更加完整的误差模型，对卫星观测值进行参数估计的一种高精度定位方法。与课堂中的广播星历单点定位相比，PPP 不再依赖广播星历和简化误差模型，而是通过双频电离层无关组合、精密产品改正、对流层估计以及模糊度参数估计，实现显著高于普通单点定位的精度。

本次程序采用 GPS 双频静态 PPP 思路。观测数据来源为 SUWN 站 RINEX 2.11 文件，配套产品包括 WHU 精密轨道 SP3、WHU 精密钟差 CLK、I20.ATX 天线文件和 WHU OSB/BIA 偏差文件。程序的主流程可概括为：观测读取 → 偏差改正 → 周跳探测 → Hatch 平滑 → 精密产品插值 → PPP 观测方程构建 → 卡尔曼滤波估计 → 收敛后稳定历元筛选 → 结果误差评估。

1.1 双频电离层无关组合原理

为消除一阶电离层延迟，程序首先构造双频电离层无关（IF）组合。若记 P_1 、 P_2 为双频伪距观测， L_1 、 L_2 为双频载波相位观测，则有：

$$P_{IF} = \alpha \cdot P_1 + \beta \cdot P_2 \quad (1)$$

$$L_{IF} = \alpha \cdot L_1 + \beta \cdot L_2 \quad (2)$$

其中，组合系数为：

$$\alpha = f_1^2 / (f_1^2 - f_2^2), \quad \beta = -f_2^2 / (f_1^2 - f_2^2) \quad (3)$$

经过 IF 组合后，一阶电离层延迟被消除，但组合后的观测噪声会放大，因此在后续滤波中需要结合合理的权阵设计和异常历元剔除策略，才能兼顾精度与稳健性。

1.2 PPP 观测方程与状态参数

静态 PPP 的核心是利用精密产品建立码观测和相位观测方程。对于 IF 组合后的观测，可以写成：

$$P_{IF} = \rho + c(dt_r - dt_s) + T + \varepsilon_P \quad (4)$$

$$L_{IF} = \rho + c(dt_r - dt_s) + T + N_{IF} + \delta_{wu} + \varepsilon_L \quad (5)$$

式中， ρ 为几何距离； dt_r 为接收机钟差； dt_s 为卫星钟差； T 为对流层延迟； N_{IF} 为电离层无关模糊度参数； δ_{wu} 为 phase wind-up 改正； ε_P 与 ε_L 分别表示码观测和相位观测误差。

本程序采用的状态向量写为：

$$x = [X, Y, Z, dt_r, ZWD, N_1, N_2, \dots, N_n]^T \quad (6)$$

其中 X 、 Y 、 Z 为测站三维坐标； dt_r 为接收机钟差； ZWD 为天顶湿延迟； N_i 为每颗卫星对应的 IF 模糊度。由于本项目为静态定位，坐标参数在时间上变化极小，因此对应过程噪声取很小；接收机钟差按历元重置更灵活，而湿延迟作为随机游走参数进行估计。

1.3 精密轨道、精密钟差与发射时刻插值

PPP 与普通单点定位最本质的区别之一，是采用精密轨道和精密钟差替代广播星历。程序通过读取 SP3 文件获取离散历元下的卫星位置，通过 CLK 文件获取离散历元下的卫星钟差。考虑到观测方程中真正需要的是信号发射时刻的卫星位置与钟差，因此程序首先用码观测粗略估计传播时延，然后在发射时刻对轨道和钟差进行插值。

在轨道插值中，程序采用 9 点 Lagrange 插值；在钟差插值中，采用线性插值。由于信号在传播过程中地球仍在自转，因此程序还对卫星坐标施加了地球自转改正（Sagnac correction），以保证发射时刻和接收时刻坐标参考框架的一致性。

1.4 对流层延迟、天线改正与偏差改正

对流层模型采用 Saastamoinen 模型与 Niell 映射函数的组合。静力部分根据测站气压和纬度模型直接计算；湿延迟部分则作为状态参数 ZWD 在滤波中估计。由于高度角越低的卫星对流层映射因子越大，因此程序中对低高度角卫星进行了更强的观测降权。

天线改正方面，程序从 ATX 文件中提取接收机天线在 L1/L2 下的 PC0，并组合为 IF PC0，用于构造实际观测点坐标。偏差改正方面，程序读取 Bias-SINEX 格式的 OSB/BIA 文件，对 C1 / P2 / L1 / L2 观测量施加卫星码偏差和相位偏差改正。通过这些改正，可以减少观测模型中的系统误差，提高最终定位精度。

1.5 周跳探测、Hatch 平滑与稳健后处理

由于载波相位观测精度高但存在周跳风险，因此程序在 PPP 之前利用 Geometry-Free 组合、Melbourne-Wubben 组合以及 LLI 锁定指示联合探测周跳；一旦探测到周跳，则重新初始化对应卫星模糊度参数。为了降低码观测噪声，程序还引入 Hatch 滤波，对伪距进行载波辅助平滑，从而改善 IF 码观测质量。

在最终结果提取阶段，程序并不对全时段历元简单取平均，而是先跳过初始收敛段，再基于移动标准差和残差 RMS 挑选出波动最小的一簇稳定历元，最后通过中值、MAD 稳健剔除和加权平均得到最终坐标。该策略可以有效降低异常历元对最终结果的影响，是本次程序能从米级提高到分米级的重要原因之一。

二、代码

本工程采用模块化设计。主程序为 run_ppp_suwn.m，统一由 ppp_config_suwn.m 管理全部输入文件路径和算法参数。按照功能划分，整个工程分为四个目录：io（数据读取）、models（模型改正与预处理）、solver（PPP 滤波求解）和 utils（工具函数）。这样的结构便于代码调试，也便于后续扩展到多系统 PPP、PPP-AR 或动态 PPP。

在实现层面，主程序的执行链条为：读取观测文件与精密产品 → 读取 BIA/OSB 并施加偏差改正 → 进行周跳探测和 Hatch 平滑 → 调用静态 PPP 滤波器建立观测方程与执行卡尔曼滤波 → 对稳定历元进行筛选和稳健统计 → 输出最终结果并与标准坐标比较。下面给出全部代码文件的完整内容。

2.1 代码文件：io/read_atx_igs.m

以下为文件 read_atx_igs.m 的完整代码内容。

```
1: % =====
2: % 文件名: read_atx_igs.m
3: % 功能: 读取 ANTEX 天线文件(ATX), 提取接收机天线 PCO 信息。
4: % 说明:
5: %   当前版本主要提取接收机天线在 GPS L1/L2 下的 PCO,
6: %   并组合成电离层无关(IF) PCO, 供接收机天线相位中心改正使用。
7: %
8: % 输入:
9: %   filepath      : ATX 文件路径
10: %   antenna_type  : 接收机天线型号字符串
11: %
12: % 输出:
13: %   atx
14: %       atx.receiver_pco_neu_mm    - 原始 PCO(mm)
15: %       atx.receiver_if_pco_neu_m  - IF 组合后的 PCO(m)
16: % =====
17: function atx = read_atx_igs(filepath, antenna_type, ~)
18: fid = fopen(filepath, 'r');
19: if fid < 0
20:     error('无法打开 ATX 文件: %s', filepath);
21: end
22: cleanup = onCleanup(@() fclose(fid));
23:
24: % 当前站使用的接收机天线型号
```

```

25: anttype = strtrim(antenna_type);
26: % 接收机天线在 L1/L2 下的 PCO, NEU 顺序, 单位 mm
27: rcv_pco = nan(2,3); % G01/G02 -> N E U (mm)
28: % GPS 卫星天线 PCO 预留矩阵, 当前主要用于读取保存

29: sat_pco = nan(32,2,3); % prn, freq, neu(mm)
30: in_block = false;
31: cur_type = '';
32: cur_sat = 0;
33: cur_is_rcv = false;
34: cur_freq = 0;
35:
36: while true
37:     line = fgetl(fid);
38:     if ~ischar(line)
39:         break
40:     end
41:     line = pad_line(line, 80);
42:     label = strtrim(line(61:80));
43:     switch label
44:         case 'START OF ANTENNA'
45:             in_block = true;
46:             cur_type = '';
47:             cur_sat = 0;
48:             cur_is_rcv = false;
49:             cur_freq = 0;
50:         case 'TYPE / SERIAL NO'
51:             if ~in_block, continue; end
52:             lhs = strtrim(line(1:20));
53:             syschar = strtrim(line(21));
54:             if startsWith(lhs, 'G') && numel(lhs) >= 3 && all(isstrprop(lhs(2:3),'digit'))
55:                 cur_is_rcv = false;
56:                 cur_sat = str2double(lhs(2:3));
57:                 cur_type = lhs;
58:             elseif strcmp(strtrim(line(1:20)), anttype)
59:                 cur_is_rcv = true;
60:                 cur_sat = 0;
61:                 cur_type = anttype;
62:             elseif strcmp(syschar, 'G') && all(isstrprop(strtrim(line(22:23)), 'digit'))
63:                 cur_is_rcv = false;
64:                 cur_sat = str2double(strtrim(line(22:23)));
65:             end
66:         case 'START OF FREQUENCY'
67:             freq_code = strtrim(line(4:6));
68:             if strcmp(freq_code, 'G01')
69:                 cur_freq = 1;
70:             elseif strcmp(freq_code, 'G02')
71:                 cur_freq = 2;
72:             else
73:                 cur_freq = 0;
74:             end
75:         case 'NORTH / EAST / UP'
76:             if cur_freq == 0
77:                 continue
78:             end
79:             vals = sscanf(line(1:60), '%f');
80:             if numel(vals) >= 3
81:                 if cur_is_rcv
82:                     rcv_pco(cur_freq,:) = vals(1:3)';
83:                 elseif cur_sat >= 1 && cur_sat <= 32
84:                     sat_pco(cur_sat,cur_freq,:) = vals(1:3);
85:                 end
86:             end
87:         case 'END OF ANTENNA'
88:             in_block = false;
89:     end
90: end
91:
92: atx.receiver_pco_neu_mm = rcv_pco;
93: atx.sat_pco_neu_mm = sat_pco;

```

```

94: atx.antenna_type = anttype;
95: % 将 L1/L2 PCO 合成为 IF PCO, 并由 mm 转为 m
96: atx.receiver_if_pco_neu_m = iono_free_pco(rcv_pco) ./ 1000;
97: end
98:
99: function ifpco = iono_free_pco(pco)
100: f = gps_frequencies();
101: a1 = f.f1^2 / (f.f1^2 - f.f2^2);
102: a2 = -f.f2^2 / (f.f1^2 - f.f2^2);
103: ifpco = a1 * pco(1,:) + a2 * pco(2,:);
104: end
105:
106: function s = pad_line(s,n)
107: if numel(s) < n
108:     s = [s repmat(' ',1,n-numel(s))];
109: end
110: end

```

2.2 代码文件: io/read_bia_osb.m

以下为文件 read_bia_osb.m 的完整代码内容。

```

1: % =====
2: % 文件名: read_bia_osb.m
3: % 功能: 读取 Bias-SINEX 格式的 BIA/OSB 文件。
4: % 说明:
5: %   当前版本主要针对 GPS 读取:
6: %       C1C / C2W / C2L / L1C(or L1W) / L2W / L2L
7: %   结果统一转换为米, 用于码偏差与相位偏差改正。
8: %
9: % 输入:
10: %   filepath : BIA 文件路径
11: %
12: % 输出:
13: %   bia
14: %       bia.code - GPS 码偏差(米)
15: %       bia.phase - GPS 相位偏差(米)
16: % =====
17: function bia = read_bia_osb(filepath)
18: fid = fopen(filepath, 'r');
19: if fid < 0
20:     error('无法打开 BIA 文件: %s', filepath);
21: end
22: cleanup = onCleanup(@() fclose(fid));
23:
24: % 光速, 用于将 ns 偏差换算为米
25: c = 299792458.0;
26: bia.file = filepath;
27: bia.code = nan(32,3); % [C1C C2W C2L] in m
28: bia.phase = nan(32,3); % [L1C/L1W L2W L2L] in m
29:
30: while true
31:     line = fgetl(fid);
32:     if ~ischar(line)
33:         break
34:     end
35:     if numel(line) < 80
36:         continue
37:     end
38:     if ~strcmp(line(2:4), 'SB ')
39:         % no-op
40:     end
41:     % 仅处理 GPS 卫星的 OSB 记录
42:     if ~startsWith(line, ' OSB G')
43:         continue
44:     end

```

```

45:
46:     prn = str2double(strtrim(line(11:13)));
47:     obs1 = strtrim(line(24:26));
48:     val = sscanf(line(71:end), '%f', 1);
49:     if isnan(prn) || prn < 1 || prn > 32 || isempty(val)
50:         continue
51:     end
52:     % BIA 文件给出的单位为 ns, 这里统一换算为米
53:     val_m = val * 1e-9 * c;
54:
55:     switch obs1
56:     case 'C1C'
57:         bia.code(prn,1) = val_m;
58:     case 'C2W'
59:         bia.code(prn,2) = val_m;
60:     case 'C2L'
61:         bia.code(prn,3) = val_m;
62:     case {'L1C','L1W'}
63:         bia.phase(prn,1) = val_m;
64:     case 'L2W'
65:         bia.phase(prn,2) = val_m;
66:     case 'L2L'
67:         bia.phase(prn,3) = val_m;
68:     end
69: end
70: end

```

2.3 代码文件: io/read_clk_whu.m

以下为文件 read_clk_whu.m 的完整代码内容。

```

1: % =====
2: % 文件名: read_clk_whu.m
3: % 功能: 读取 WHU 精密钟差文件(.CLK)。
4: % 说明:
5: %   提取 GPS 卫星 AS 记录中的卫星钟差, 以秒为单位保存。
6: %   后续 PPP 中通过 sat_clock_interp.m 进行历元插值使用。
7: %
8: % 输入:
9: %   filepath : CLK 文件路径
10: %
11: % 输出:
12: %   clk      : 钟差结构体
13: %       clk.sod - 钟差历元时间序列(秒)
14: %       clk.bias - 各卫星钟差(秒)
15: %       clk.interval - 时钟采样间隔(秒)
16: % =====
17: function clk = read_clk_whu(filepath, ~)
18: fid = fopen(filepath, 'r');
19: if fid < 0
20:     error('无法打开 CLK 文件: %s', filepath);
21: end
22: cleanup = onCleanup(@() fclose(fid));
23:
24: % 存放所有钟差记录的历元时刻
25: records_t = [];
26: records_c = [];
27: base_day = NaN;
28: while true
29:     line = fgetl(fid);
30:     if ~ischar(line)
31:         break
32:     end
33:     if numel(line) < 40
34:         continue
35:     end

```

```

36: % 仅处理 GPS 卫星钟差记录
37: if startsWith(line, 'AS G')
38:     prn = str2double(line(5:7));
39:     yy = str2double(strtrim(line(9:12)));
40:     mo = str2double(strtrim(line(14:15)));
41:     dd = str2double(strtrim(line(17:18)));
42:     hh = str2double(strtrim(line(20:21)));
43:     mi = str2double(strtrim(line(23:24)));
44:     ss = str2double(strtrim(line(26:35)));
45:     val = sscanf(line(41:end), '%f');
46:     if isempty(val)
47:         continue
48:     end
49:     if isnan(base_day)
50:         base_day = datenum(yy,mo,dd);
51:     end
52:     % 将跨天日期统一转换为相对于基准日的秒偏移
53:     day_offset = (datenum(yy,mo,dd) - base_day) * 86400;
54:     sod = day_offset + hh*3600 + mi*60 + ss;
55:     records_t(end+1,1) = sod; %#ok<AGROW>
56:     records_c(end+1,1) = prn; %#ok<AGROW>
57:     records_c(end,2) = val(1); %#ok<AGROW>
58: end
59: end
60:
61: t = unique(records_t);
62: clk.sod = t;
63: clk.bias = nan(numel(t),32);
64: for i = 1:size(records_c,1)
65:     it = find(abs(t-records_t(i))<1e-6,1);
66:     % 将该历元该卫星的钟差写入矩阵
67:     clk.bias(it, records_c(i,1)) = records_c(i,2);
68: end
69: clk.interval = median(diff(t));
70: end

```

2.4 代码文件：io/read_rinex2_obs_gps.m

以下为文件 read_rinex2_obs_gps.m 的完整代码内容。

```

1: % =====
2: % 文件名: read_rinex2_obs_gps.m
3: % 功能: 读取 RINEX 2.xx 观测文件, 提取 GPS 双频观测。
4: % 说明:
5: % 当前版本针对你的数据格式进行适配, 主要读取:
6: %     C1 / P2 / L1 / L2
7: % 并保存每个历元的卫星列表、观测值、LLI(锁定指示)等信息。
8: %
9: % 输入:
10: %   filepath : 观测文件完整路径
11: %   cfg       : 配置结构体
12: %
13: % 输出:
14: %   obs       : 观测数据结构体
15: %       obs.header - 文件头信息
16: %       obs.epochs - 每个历元的观测记录
17: %       obs.gps_prns - 文件中出现过的 GPS 卫星编号
18: %
19: % 重要说明:
20: % 1) 本函数只对 GPS 卫星进行解算数据提取;
21: % 2) 观测值阶段暂不进行几何改正, 仅完成文件解析与存储;
22: % 3) 若需要扩展到 GLO/GAL/BDS, 可在此函数继续扩展。
23: % =====
24: function obs = read_rinex2_obs_gps(filepath, cfg)
25: % 打开观测文件

```

```

26: fid = fopen(filepath, 'r');
27: if fid < 0
28:     error('无法打开 OBS 文件: %s', filepath);
29: end
30: % 使用 onCleanup 保证函数退出时自动关闭文件
31: cleanup = onCleanup(@() fclose(fid));
32:
33: % 用于保存观测文件头中的测站、接收机、天线、时间系统等信息
34: header = struct();
35: % 保存当前观测文件头声明的观测值类型列表
36: obs_types = {};
37:
38: % ----- 1. 读取文件头 -----
39: while true
40:     line = fgetl(fid);
41:     if ~ischar(line)
42:         error('OBS 文件头未找到 END OF HEADER');
43:     end
44:
45:     % RINEX 标签位于 61:80 列, 先补足到 80 列再取标签
46:     line80 = pad_line(line,80);
47:     tag = strtrim(line80(61:80));
48:
49:     switch tag
50:     case 'RINEX VERSION / TYPE'
51:         % 读取 RINEX 版本、文件类型、系统类型
52:         header.version = str2double(strtrim(line80(1:9)));
53:         header.file_type = strtrim(line80(21));
54:         header.sys = strtrim(line80(41));
55:
56:     case 'MARKER NAME'
57:         % 站点名称
58:         header.marker_name = strtrim(line80(1:60));
59:
60:     case 'REC # / TYPE / VERS'
61:         % 接收机型号信息
62:         header.receiver = strtrim(line80(21:40));
63:
64:     case 'ANT # / TYPE'
65:         % 天线型号信息
66:         header.antenna_type = strtrim(line80(21:40));
67:
68:     case 'APPROX POSITION XYZ'
69:         % 文件头中的近似坐标
70:         header.approx_xyz = sscanf(line80(1:60), '%f', [3,1]);
71:
72:     case 'ANTENNA: DELTA H/E/N'
73:         % 天线高/偏心, 顺序是 H E N
74:         header.ant_delta_hen = sscanf(line80(1:60), '%f', [3,1]);
75:
76:     case '# / TYPES OF OBSERV'
77:         % 读取观测类型总数
78:         nob = sscanf(line80(1:6), '%d', 1);
79:         vals = cell(1,nob);
80:         idx = 1;
81:         cur = line80;
82:
83:         % RINEX2 中观测类型可能跨多行, 需要连续读取
84:         while idx <= nob
85:             for k = 1:9
86:                 c1 = 10 + (k-1)*6 + 1;
87:                 c2 = c1 + 1;
88:                 if c2 <= numel(cur)
89:                     token = strtrim(cur(c1:c2));
90:                     if ~isempty(token)
91:                         vals{idx} = token;
92:                         idx = idx + 1;

```

```

93:                     if idx > nobs
94:                         break;
95:                     end
96:                 end
97:             end
98:         end
99:         if idx <= nobs
100:             nxt = fgetl(fid);
101:             if ~ischar(nxt)
102:                 error('OBS 文件头在读取观测类型时意外结束');
103:             end
104:             cur = pad_line(nxt,80);
105:         end
106:     end
107:     obs_types = vals;
108:
109:     case 'INTERVAL'
110:         % 采样间隔
111:         header.interval = sscanf(line80(1:10), '%f', 1);
112:
113:     case 'TIME OF FIRST OBS'
114:         % 首历元时刻
115:         vals = sscanf(line80(1:43), '%f');
116:         header.first = vals(:)';
117:
118:     case 'TIME OF LAST OBS'
119:         % 末历元时刻
120:         vals = sscanf(line80(1:43), '%f');
121:         header.last = vals(:)';
122:
123:     case 'LEAP SECONDS'
124:         % 闰秒信息
125:         header.leap_seconds = sscanf(line80(1:6), '%f', 1);
126:
127:     case 'END OF HEADER'
128:         % 文件头结束
129:         break
130:     end
131: end
132:
133: % 将观测类型名称映射到索引位置，后续便于快速取值
134: map = make_obs_type_map(obs_types);
135: % 预分配历元结构体数组
136: epochs = struct('time', {}, 'sod', {}, 'sat', {}, 'C1', {}, 'P2', {}, 'L1', {}, 'L2', {}, 'l1l1', {},
'l1l2', {});
137: % 记录文件中出现过的 GPS 卫星编号
138: gps_seen = false(1,32);
139:
140: % ----- 2. 读取观测文件体 -----
141: while true
142:     line = fgetl(fid);
143:     if ~ischar(line)
144:         break
145:     end
146:     % 跳过空行
147:     if isempty(strtrim(line))
148:         continue
149:     end
150:
151:     line80 = pad_line(line, 80);
152:
153:     % 解析历元时间与历元头信息
154:     yr = sscanf(line80(1:3), '%d', 1);
155:     if isempty(yr)
156:         continue
157:     end
158:     mo = sscanf(line80(4:6), '%d', 1);

```

```

159:     dy = sscanf(line80(7:9), '%d', 1);
160:     hr = sscanf(line80(10:12), '%d', 1);
161:     mi = sscanf(line80(13:15), '%d', 1);
162:     se = sscanf(line80(16:26), '%f', 1);
163:     flag = sscanf(line80(27:29), '%d', 1);
164:     nsat = sscanf(line80(30:32), '%d', 1);
165:
166:     if isempty(flag) || isempty(nsat)
167:         continue
168:     end
169:
170:     % 两位年份展开为四位年份
171:     if yr < 80
172:         yr = yr + 2000;
173:     else
174:         yr = yr + 1900;
175:     end
176:
177:     % 若不是正常观测历元，则跳过该历元的后续卫星列表续行
178:     if flag ~= 0
179:         extra_sat_lines = max(0, ceil((nsat - 12) / 12));
180:         for ii = 1:extra_sat_lines
181:             fgetl(fid);
182:         end
183:         continue
184:     end
185:
186:     % 将历元时间转为 MATLAB datenum 形式
187:     epoch_dt = datenum(yr,mo,dy,hr,mi,se);
188:     % 保存历元对应的当天秒
189:     sod = hr*3600 + mi*60 + se;
190:
191:     % 存储当前历元广播的卫星号字符串，例如 G01、G23
192:     sat_ids = cell(1, nsat);
193:
194:     % RINEX2 历元头一行的卫星列表位于 33:68，最多 12 颗卫星
195:     cols = line80(33:68);
196:
197:     % 若卫星数大于 12，则继续读取卫星列表续行
198:     extra_sat_lines = max(0, ceil((nsat - 12) / 12));
199:     for ii = 1:extra_sat_lines
200:         nxt = fgetl(fid);
201:         if ~ischar(nxt)
202:             break
203:         end
204:         nxt80 = pad_line(nxt,80);
205:         % 注意：续行中真正的卫星列表仍然从 33 列开始，而不是整行拼接
206:         cols = [cols nxt80(33:68)]; %#ok<AGROW>
207:     end
208:
209:     % 将连续的卫星号字符串按每 3 列拆分
210:     for k = 1:nsat
211:         i1 = (k-1)*3 + 1;
212:         i2 = k*3;
213:         if i2 <= numel(cols)
214:             sat_ids{k} = strtrim(cols(i1:i2));
215:         else
216:             sat_ids{k} = '';
217:         end
218:     end
219:
220:     sat = [];
221:     C1 = []; P2 = []; L1 = []; L2 = []; lli1 = []; lli2 = [];
222:     % 一个卫星的观测值占多少行，取决于观测类型总数
223:     nlines_obs = ceil(numel(obs_types)/5);
224:

```

```

225: % 逐颗卫星解析本历元观测值
226: for k = 1:nsat
227:     sid = sat_ids{k};
228:
229:     % 一个卫星可能占多行观测值，这里先全部读入
230:     blocks = cell(nlines_obs,1);
231:     for b = 1:nlines_obs
232:         raw = fgetl(fid);
233:         if ~ischar(raw)
234:             raw = '';
235:         end
236:         blocks{b} = pad_line(raw,80);
237:     end
238:
239:     % 保护判断：若卫星号长度不足 3，则直接跳过，避免 sid(2:3) 越界
240:     if numel(sid) < 3
241:         continue
242:     end
243:     % 当前工程仅处理 GPS 卫星
244:     if sid(1) ~= 'G'
245:         continue
246:     end
247:
248:     % 提取 GPS PRN 号
249:     prn = str2double(sid(2:3));
250:     if isnan(prn) || prn < 1 || prn > 32
251:         continue
252:     end
253:     gps_seen(prn) = true;
254:
255:     % 保存该卫星当前历元所有观测类型的数值
256:     vals = nan(1, numel(obs_types));
257:     llis = nan(1, numel(obs_types));
258:     idx = 1;
259:     for b = 1:nlines_obs
260:         cur = blocks{b};
261:         for j = 1:5
262:             if idx > numel(obs_types)
263:                 break
264:             end
265:             % 每个观测值固定宽度为 16 列，其中前 14 列是观测值
266:             p1 = (j-1)*16 + 1;
267:             p2 = p1 + 13;
268:             if p2 <= numel(cur)
269:                 token = strtrim(cur(p1:p2));
270:                 if ~isempty(token)
271:                     vals(idx) = str2double(token);
272:                 end
273:                 % 第 15 列一般可视为 LLI 字段
274:                 lli_col = p1 + 14;
275:                 if lli_col <= numel(cur)
276:                     lli_token = strtrim(cur(lli_col));
277:                     if ~isempty(lli_token)
278:                         llis(idx) = str2double(lli_token);
279:                     end
280:                 end
281:             end
282:             idx = idx + 1;
283:         end
284:     end
285:
286:     % 根据观测类型映射，提取 GPS 解算所需的 C1 / P2 / L1 / L2
287:     c1 = get_obs_value(vals, map, {'C1','P1'});
288:     p2 = get_obs_value(vals, map, {'P2','C2'});
289:     l1 = get_obs_value(vals, map, {'L1'});
290:     l2 = get_obs_value(vals, map, {'L2'});
291:     i1 = get_obs_llis(llis, map, {'L1'});
292:     i2 = get_obs_llis(llis, map, {'L2'});

```

```

293:
294:     sat(end+1,1) = prn; %#ok<AGROW>
295:     C1(end+1,1) = c1; %#ok<AGROW>
296:     P2(end+1,1) = p2; %#ok<AGROW>
297:     L1(end+1,1) = l1; %#ok<AGROW>
298:     L2(end+1,1) = l2; %#ok<AGROW>
299:     lli1(end+1,1) = i1; %#ok<AGROW>
300:     lli2(end+1,1) = i2; %#ok<AGROW>
301: end
302:
303: % 将本历元最终解析结果写入 epochs 结构体
304: epochs(end+1).time = epoch_dt; %#ok<AGROW>
305: epochs(end).sod = sod;
306: epochs(end).sat = sat;
307: epochs(end).C1 = C1;
308: epochs(end).P2 = P2;
309: epochs(end).L1 = L1;
310: epochs(end).L2 = L2;
311: epochs(end).lli1 = lli1;
312: epochs(end).lli2 = lli2;
313: end
314:
315: % 汇总输出
316: obs.header = header;
317: obs.obs_types = obs_types;
318: obs.epochs = epochs;
319: obs.gps_prns = find(gps_seen);
320: obs.freq = gps_frequencies();
321: obs.cfg = cfg;
322: end
323:
324: function s = pad_line(s, n)
325: % 将字符串右侧补空格到固定长度，方便按固定列读取
326: if ~ischar(s)
327:     s = char(s);
328: end
329: if numel(s) < n
330:     s = [s repmat(' ',1,n-numel(s))];
331: end
332: end
333:
334: function map = make_obs_type_map(obs_types)
335: % 构建 观测类型 -> 索引 的映射表
336: map = containers.Map();
337: for i = 1:numel(obs_types)
338:     map(obs_types{i}) = i;
339: end
340: end
341:
342: function v = get_obs_value(vals, map, names)
343: % 按优先级从多个观测类型名称中提取第一个可用观测值
344: v = NaN;
345: for i = 1:numel(names)
346:     if isKey(map, names{i})
347:         idx = map(names{i});
348:         if idx <= numel(vals)
349:             v = vals(idx);
350:             if ~isnan(v) && v ~= 0
351:                 return
352:             end
353:         end
354:     end
355: end
356: end
357:
358: function v = get_obs_lli(vals, map, names)
359: % 提取指定观测类型对应的 LLI 值
360: v = NaN;

```

```

361: for i = 1:numel(names)
362:     if isKey(map, names{i})
363:         idx = map(names{i});
364:         if idx <= numel(vals)
365:             v = vals(idx);
366:             return
367:         end
368:     end
369: end
370: end

```

2.5 代码文件：io/read_sp3_multi.m

以下为文件 read_sp3_multi.m 的完整代码内容。

```

1: % =====
2: % 文件名: read_sp3_multi.m
3: % 功能: 读取一个或多个 SP3 精密轨道文件并拼接。
4: % 说明:
5: % 本函数允许同时输入前一天/当天/后一天 SP3 文件, 最终统一到同一时间轴。
6: % 在后续轨道插值时, 可直接使用拼接后的轨道矩阵。
7: %
8: % 输入:
9: %   file_list : SP3 文件路径元胞数组
10: %
11: % 输出:
12: %   sp3       : 轨道结构体
13: %       sp3.sod      - 轨道历元秒序列
14: %       sp3.pos      - 卫星精密坐标(米)
15: %       sp3.interval - 轨道采样间隔(秒)
16: % =====
17: function sp3 = read_sp3_multi(file_list, ~)
18: % 保存所有 SP3 文件拼接后的时间轴
19: all_t = [];
20: all_pos = nan(0,32,3);
21: base_dn = [];
22: for i = 1:numel(file_list)
23:     one = read_sp3_one(file_list{i});
24:     if isempty(base_dn)
25:         base_dn = one.day_dn;
26:     end
27:     % 将不同日期的 SP3 时间轴统一到同一参考天
28:     offset = (one.day_dn - base_dn) * 86400;
29:     all_t = [all_t; one.sod(:) + offset]; %#ok<AGROW>
30:     all_pos = cat(1, all_pos, one.pos);
31: end
32: [all_t, ia] = unique(all_t, 'stable');
33: all_pos = all_pos(ia, :, :);
34: sp3.sod = all_t;
35: sp3.pos = all_pos;
36: % 轨道采样间隔通常由相邻时间差中值得到
37: sp3.interval = median(diff(all_t));
38: end
39:
40: function sp3 = read_sp3_one(filepath)
41: fid = fopen(filepath, 'r');
42: if fid < 0
43:     error('无法打开 SP3 文件: %s', filepath);
44: end
45: cleanup = onCleanup(@( ) fclose(fid));
46:
47: pos = nan(288,32,3);
48: sod = nan(288,1);
49: current_idx = 0;
50: base_day = NaN;
51: while true

```

```

52:     line = fgetl(fid);
53:     if ~ischar(line)
54:         break
55:     end
56:     if startsWith(line, '*')
57:         vals = sscanf(line(2:end), '%f');
58:         yy = vals(1); mm = vals(2); dd = vals(3);
59:         hh = vals(4); mi = vals(5); ss = vals(6);
60:         if isnan(base_day)
61:             base_day = datenum(yy,mm,dd);
62:         end
63:         current_idx = current_idx + 1;
64:         sod(current_idx) = hh*3600 + mi*60 + ss;
65:     elseif startsWith(line, 'PG')
66:         prn = str2double(line(3:4));
67:         vals = sscanf(line(5:end), '%f');
68:         if prn >= 1 && prn <= 32 && current_idx >= 1 && numel(vals) >= 3
69:             pos(current_idx,prn,1:3) = vals(1:3) * 1000;
70:         end
71:     end
72: end
73: pos = pos(1:current_idx,:,:)
74: sod = sod(1:current_idx);
75: sp3.sod = sod;
76: sp3.pos = pos;
77: sp3.day_dn = base_day;
78: end

```

2.6 代码文件：models/apply_bia_osb.m

以下为文件 apply_bia_osb.m 的完整代码内容。

```

1: % =====
2: % 文件名: apply_bia_osb.m
3: % 功能: 将 OSB/BIA 偏差改正应用到观测值。
4: % 说明:
5: %   对 C1 / P2 / L1 / L2 对应观测值进行逐卫星偏差改正。
6: %   对于相位偏差, 先由米转换为周数, 再修正到 L1/L2 观测量中。
7: %
8: % 输入:
9: %   obs : 原始观测结构体
10: %   bia : 由 read_bia_osb.m 解析后的偏差结构体
11: %
12: % 输出:
13: %   obs : 加偏差改正后的观测结构体
14: % =====
15: function obs = apply_bia_osb(obs, bia)
16: for i = 1:numel(obs.epochs)
17:     ep = obs.epochs(i);
18:     ns = numel(ep.sat);
19:     for k = 1:ns
20:         prn = ep.sat(k);
21:         if prn < 1 || prn > 32
22:             continue
23:         end
24:
25:         if ~isnan(ep.C1(k)) && ~isnan(bia.code(prn,1))
26:             % 对 C1 码观测减去对应卫星 OSB 偏差
27:             ep.C1(k) = ep.C1(k) - bia.code(prn,1);
28:         end
29:
30:         if ~isnan(ep.P2(k))
31:             if ~isnan(bia.code(prn,2))
32:                 ep.P2(k) = ep.P2(k) - bia.code(prn,2);
33:             elseif ~isnan(bia.code(prn,3))
34:                 ep.P2(k) = ep.P2(k) - bia.code(prn,3);
35:             end

```

```

36:         end
37:
38:         % phase OSB 在 float PPP 中大多被模糊度吸收，先做一致性改正
39:         if ~isnan(ep.L1(k)) && ~isnan(bia.phase(prn,1))
40:             ep.L1(k) = ep.L1(k) - bia.phase(prn,1) / (gps_frequencies().c / gps_frequencies().f1);
41:         end
42:
43:         if ~isnan(ep.L2(k))
44:             if ~isnan(bia.phase(prn,2))
45:                 ep.L2(k) = ep.L2(k) - bia.phase(prn,2) / (gps_frequencies().c / gps_frequencies().f2);
46:             elseif ~isnan(bia.phase(prn,3))
47:                 ep.L2(k) = ep.L2(k) - bia.phase(prn,3) / (gps_frequencies().c / gps_frequencies().f2);
48:             end
49:         end
50:     end
51:     obs.epochs(i) = ep;
52: end
53: end

```

2.7 代码文件：models/detect_cycle_slip_basic.m

以下为文件 detect_cycle_slip_basic.m 的完整代码内容。

```

1: % =====
2: % 文件名: detect_cycle_slip_basic.m
3: % 功能: 基础周跳探测。
4: % 说明:
5: %   采用以下信息联合判断:
6: %       1) Geometry-Free 组合
7: %       2) Melbourne-Wubben 组合
8: %       3) LLI 锁定指示
9: %   若发生异常，则将该历元对应卫星记为 slip。
10: % =====
11: function obs = detect_cycle_slip_basic(obs)
12: f = gps_frequencies();
13: % 计算 L1 波长
14: lam1 = f.c / f.f1;
15: % 计算 L2 波长
16: lam2 = f.c / f.f2;
17: prev = containers.Map('KeyType','char','ValueType','any');
18:
19: for i = 1:numel(obs.epochs)
20:     ep = obs.epochs(i);
21:     slip = false(size(ep.sat));
22:     for k = 1:numel(ep.sat)
23:         prn = ep.sat(k);
24:         key = sprintf('G%02d', prn);
25:         if isnan(ep.L1(k)) || isnan(ep.L2(k)) || isnan(ep.C1(k)) || isnan(ep.P2(k))
26:             % 一旦判断异常，则将该卫星该历元记为周跳
27:             slip(k) = true;
28:             continue
29:         end
30:         % Geometry-Free 组合，对周跳和电离层变化敏感
31:         gf = ep.L1(k)*lam1 - ep.L2(k)*lam2;
32:         % Melbourne-Wubben 组合，用于辅助周跳探测
33:         mw = melbourne_wubben(ep.C1(k), ep.P2(k), ep.L1(k), ep.L2(k), f);
34:         lli_hit = (~isnan(ep.l1l1(k)) && ep.l1l1(k) > 0) || (~isnan(ep.l1l2(k)) && ep.l1l2(k) > 0);
35:         if isKey(prev, key)
36:             old = prev(key);
37:             if abs(gf - old.gf) > 0.08 || abs(mw - old.mw) > 4 || lli_hit
38:                 % 一旦判断异常，则将该卫星该历元记为周跳
39:                 slip(k) = true;
40:             end
41:         else
42:             % 一旦判断异常，则将该卫星该历元记为周跳

```

```

43:         slip(k) = true;
44:     end
45:     prev(key) = struct('gf',gf,'mw',mw);
46: end
47: obs.epochs(i).slip = slip;
48: end
49: end
50:
51: function % Melbourne-Wubbena 组合, 用于辅助周跳探测
52: mw = melbourne_wubben(C1, P2, L1, L2, f)
53: phi_wl = (f.c/f.f1)*L1 - (f.c/f.f2)*L2;
54: code_nw = (f.f1*C1 + f.f2*P2) / (f.f1 + f.f2);
55: lam_wl = f.c/(f.f1-f.f2);
56: mw = (phi_wl - code_nw)/lam_wl;
57: end

```

2.8 代码文件: models/hatch_smooth_obs.m

以下为文件 hatch_smooth_obs.m 的完整代码内容。

```

1: % =====
2: % 文件名: hatch_smooth_obs.m
3: % 功能: Hatch 滤波平滑伪距。
4: % 说明:
5: %   使用载波相位的高精度短时稳定特性对伪距进行平滑,
6: %   改善码噪声, 生成每颗卫星每历元的 IF 平滑伪距 P_if_sm。
7: % =====
8: function obs = hatch_smooth_obs(obs, window)
9: f = gps_frequencies();
10: % 计算 L1 波长
11: lam1 = f.c / f.f1;
12: % 计算 L2 波长
13: lam2 = f.c / f.f2;
14: state = containers.Map('KeyType','char','ValueType','any');
15:
16: for i = 1:numel(obs.epochs)
17:     ep = obs.epochs(i);
18:     for k = 1:numel(ep.sat)
19:         prn = ep.sat(k);
20:         key = sprintf('G%02d', prn);
21:         if ep.slip(k)
22:             remove_if_exists(state, key);
23:         end
24:         if isnan(ep.C1(k)) || isnan(ep.P2(k)) || isnan(ep.L1(k)) || isnan(ep.L2(k))
25:             continue
26:         end
27:         % 当前历元的 IF 码观测
28:         Pif = iono_free_comb(ep.C1(k), ep.P2(k), f);
29:         % 当前历元的 IF 相位观测(单位米)
30:         Lif = iono_free_comb(ep.L1(k)*lam1, ep.L2(k)*lam2, f);
31:         if isKey(state, key)
32:             st = state(key);
33:             n = min(st.n + 1, window);
34:             Psm = (Pif + (n-1) * (st.Psm + (Lif - st.Lif_prev))) / n;
35:             st.n = n;
36:             st.Psm = Psm;
37:             st.Lif_prev = Lif;
38:             state(key) = st;
39:         else
40:             st = struct('n',1,'Psm',Pif,'Lif_prev',Lif);
41:             state(key) = st;
42:         end
43:         ep.P_if_sm(k,1) = state(key).Psm;
44:     end
45:     obs.epochs(i) = ep;
46: end
47: end

```

```

48:
49: function remove_if_exists(mp, key)
50: if isKey(mp, key)
51:     remove(mp, key);
52: end
53: end

```

2.9 代码文件: models/phase_windup_simple.m

以下为文件 phase_windup_simple.m 的完整代码内容。

```

1: % =====
2: % 文件名: phase_windup_simple.m
3: % 功能: 简化版相位缠绕(phase wind-up)改正。
4: % 说明:
5: %   本函数给出一个简化实现, 用于减少相位模型中的系统偏差。
6: %   若后续要进一步追求 PPP-AR 精度, 可替换为更严格的卫星姿态模型版本。
7: % =====
8: function dphi_m = phase_windup_simple(rec_xyz, sat_xyz)
9: % 简化版风 up, 保证模型结构不同于旧代码; 如需更严谨可再升级
10: % 用 persistent 记录上一次 wind-up 值, 保证时间序列连续
11: persistent last_val
12: if isempty(last_val)
13:     last_val = containers.Map('KeyType','char','ValueType','double');
14: end
15: % 构造接收机到卫星的视线单位向量
16: u = (rec_xyz(:) - sat_xyz(:));
17: u = u / norm(u);
18: ez = sat_xyz(:) / norm(sat_xyz(:));
19: ey = cross(ez, [0;0;1]);
20: if norm(ey) < 1e-8
21:     ey = [0;1;0];
22: end
23: ey = ey / norm(ey);
24: ex = cross(ey, ez);
25:
26: [lat, lon, ~] = ecef2geodetic_simple(rec_xyz(1), rec_xyz(2), rec_xyz(3));
27: er = [cos(lat)*cos(lon); cos(lat)*sin(lon); sin(lat)];
28: en = [-sin(lat)*cos(lon); -sin(lat)*sin(lon); cos(lat)];
29: ee = [-sin(lon); cos(lon); 0];
30: Dsat = ex - u*dot(u,ex) - cross(u,ey);
31: Drec = ee - u*dot(u,ee) + cross(u,en);
32: ang = atan2(dot(u,cross(Dsat,Drec)), dot(Dsat,Drec));
33: key = 'global';
34: if isKey(last_val, key)
35:     prev = last_val(key);
36:     n = round((prev - ang)/(2*pi));
37:     ang = ang + 2*pi*n;
38: end
39: last_val(key) = ang;
40: % 将 wind-up 相位转换为近似米值(L1 波长量级)
41: dphi_m = ang / (2*pi) * 0.190293672798; % 近似转换到 L1 波长量级米值
42: end

```

2.10 代码文件: models/preprocess_obs.m

以下为文件 preprocess_obs.m 的完整代码内容。

```

1: % =====
2: % 文件名: preprocess_obs.m
3: % 功能: 观测预处理总控函数。
4: % 说明:
5: %   目前依次执行:
6: %       1) detect_cycle_slip_basic.m 周跳探测
7: %       2) hatch_smooth_obs.m      Hatch 平滑伪距

```

```

8: %
9: % 输入:
10: %   obs : 原始观测结构体
11: %   cfg : 配置结构体
12: %
13: % 输出:
14: %   obs : 预处理后的观测结构体
15: % =====
16: function obs = preprocess_obs(obs, cfg)
17: % 若开启周跳探测, 则先对原始相位观测进行周跳检查
18: if cfg.proc.enable_cycle_slip
19:     obs = detect_cycle_slip_basic(obs);
20: end
21: % 若开启 Hatch 平滑, 则对伪距进行载波辅助平滑
22: if cfg.proc.enable_hatch
23:     obs = hatch_smooth_obs(obs, cfg.proc.hatch_window);
24: end
25: end

```

2.11 代码文件: models/sat_clock_interp.m

以下为文件 sat_clock_interp.m 的完整代码内容。

```

1: % =====
2: % 文件名: sat_clock_interp.m
3: % 功能: 卫星钟差插值。
4: % 说明:
5: %   对精密钟差文件中的离散钟差历元进行线性插值,
6: %   输出目标发射时刻的卫星钟差。
7: % =====
8: function dt = sat_clock_interp(clk, t_abs, prn)
9: % 取出指定卫星所有历元的钟差序列
10: col = clk.bias(:,prn);
11: idx = find(~isnan(col));
12: if numel(idx) < 2
13:     dt = NaN;
14:     return
15: end
16: t = clk.sod(idx);
17: y = col(idx);
18: if t_abs < t(1) || t_abs > t(end)
19:     dt = NaN;
20:     return
21: end
22: % 在线性插值后得到目标时刻钟差
23: dt = interp1(t, y, t_abs, 'linear');
24: end

```

2.12 代码文件: models/sp3_interp.m

以下为文件 sp3_interp.m 的完整代码内容。

```

1: % =====
2: % 文件名: sp3_interp.m
3: % 功能: 卫星精密轨道插值。
4: % 说明:
5: %   采用 9 点 Lagrange 插值, 对目标时刻的卫星位置进行求值。
6: %   该函数为 PPP 轨道模型核心函数之一。
7: % =====
8: function sat_xyz = sp3_interp(sp3, t_abs, prn)
9: % 取出目标卫星在所有 SP3 历元下的三维坐标
10: xyz = squeeze(sp3.pos(:,prn,:));
11: valid = all(~isnan(xyz),2);
12: if nnz(valid) < 9
13:     sat_xyz = [NaN;NaN;NaN];

```

```

14:     return
15: end
16: t = sp3.sod(valid);
17: xyz = xyz(valid,:);
18: if t_abs < t(1) || t_abs > t(end)
19:     sat_xyz = [NaN;NaN;NaN];
20:     return
21: end
22: % 确定目标时刻位于哪个离散历元附近
23: n = find(t <= t_abs, 1, 'last');
24: st = max(1, n-4);
25: fn = min(numel(t), st+8);
26: st = max(1, fn-8);
27: sel = st:fn;
28: if numel(sel) < 9
29:     sat_xyz = [NaN;NaN;NaN];
30:     return
31: end
32: sat_xyz = zeros(3,1);
33: for j = 1:3
34:     % 对 X/Y/Z 三个方向分别做 Lagrange 插值
35:     sat_xyz(j) = lagrange_interp(t(sel), xyz(sel,j), t_abs);
36: end
37: end

```

2.13 代码文件: models/trop_saastamoinen.m

以下为文件 trop_saastamoinen.m 的完整代码内容。

```

1: % =====
2: % 文件名: trop_saastamoinen.m
3: % 功能: 对流层模型计算。
4: % 说明:
5: % 采用 Saastamoinen + Niell 映射函数形式, 输出:
6: %     zhd - 天顶静力延迟
7: %     m_h - 静力映射函数
8: %     m_w - 湿延迟映射函数
9: % 滤波中对湿延迟部分以 ZWD 状态参数估计。
10: % =====
11: function [zhd, m_h, m_w] = trop_saastamoinen(rec_xyz, elev_rad)
12: % 先由 ECEF 坐标求测站纬度和高程
13: [lat, ~, h] = ecef2geodetic_simple(rec_xyz(1), rec_xyz(2), rec_xyz(3));
14: lat_deg = abs(lat * 180/pi);
15:
16: % 按经验大气模型估计测站气压
17: P = 1013.25 * (1 - 2.257e-5 * max(h,0))^5.2568;
18: T = 15.0 - 6.5e-3 * max(h,0) + 273.15;
19: RH = 0.50;
20: % 计算天顶静力延迟 ZHD
21: zhd = 0.0022768 * P / (1 - 0.00266*cos(2*lat) - 0.00028*h/1e3);
22:
23: e = RH * 6.108 * exp((17.15*(T-273.15)) / (234.7 + (T-273.15)));
24: zwd0 = 0.002277 * (1255/T + 0.05) * e;
25: if isnan(zwd0)
26:     zwd0 = 0.10;
27: end
28:
29: [aht, bht, cht, awt, bwt, cwt] = niell_coeff(lat_deg);
30: sinE = max(sin(elev_rad), 0.05);
31:
32: % 计算静力映射函数
33: m_h = mapf(sinE, aht, bht, cht) + (1/sinE - mapf(sinE, 2.53e-5, 5.49e-3, 1.14e-3)) * (h/1000);
34: % 计算湿延迟映射函数
35: m_w = mapf(sinE, awt, bwt, cwt);
36:
37: if ~isfinite(m_h), m_h = 1/sinE; end

```

```

38: if ~isfinite(m_w), m_w = 1/sinE; end
39: if ~isfinite(zhd), zhd = 2.3; end
40: if ~isfinite(zwd0), zwd0 = 0.10; end %#ok<NASGU>
41: end
42:
43: function m = mapf(sinE, a, b, c)
44: m = (1 + a/(1 + b/(1 + c))) / (sinE + a/(sinE + b/(sinE + c)));
45: end
46:
47: function [ah,bh,ch,aw,bw,cw] = niell_coeff(lat_deg)
48: lat_tab = [15 30 45 60 75];
49: ah_tab = [1.2769934e-3 1.2683230e-3 1.2465397e-3 1.2196049e-3 1.2045996e-3];
50: bh_tab = [2.9153695e-3 2.9152299e-3 2.9288445e-3 2.9022565e-3 2.9024912e-3];
51: ch_tab = [62.610505e-3 62.837393e-3 63.721774e-3 63.824265e-3 64.258455e-3];
52: aw_tab = [5.8021897e-4 5.6794847e-4 5.8118019e-4 5.9727542e-4 6.1641693e-4];
53: bw_tab = [1.4275268e-3 1.5138625e-3 1.4572752e-3 1.5007428e-3 1.7599082e-3];
54: cw_tab = [4.3472961e-2 4.6729510e-2 4.3908931e-2 4.4626982e-2 5.4736038e-2];
55:
56: lat_use = min(max(lat_deg,15),75);
57: ah = interp1(lat_tab, ah_tab, lat_use, 'linear');
58: bh = interp1(lat_tab, bh_tab, lat_use, 'linear');
59: ch = interp1(lat_tab, ch_tab, lat_use, 'linear');
60: aw = interp1(lat_tab, aw_tab, lat_use, 'linear');
61: bw = interp1(lat_tab, bw_tab, lat_use, 'linear');
62: cw = interp1(lat_tab, cw_tab, lat_use, 'linear');
63: end

```

2.14 代码文件：ppp_config_suwn.m

以下为文件 ppp_config_suwn.m 的完整代码内容。

```

1: % =====
2: % 文件名: ppp_config_suwn.m
3: % 功能: PPP 工程统一配置文件。
4: % 说明:
5: %   该文件集中设置所有输入文件路径、解算控制参数、滤波参数、误差模型开关、
6: %   以及标准坐标与输出控制参数。后续换数据时，优先修改本文件。
7: %
8: % 主要配置内容:
9: %   files.*      : 输入文件路径
10: %   proc.*       : 解算时间范围、截止高度角、预处理控制参数
11: %   filter.*     : 卡尔曼滤波初始方差与过程噪声参数
12: %   model.*      : 各种模型改正开关
13: %   eval.*       : 标准坐标(参考坐标)
14: %   output.*     : 最终结果筛选、加权、作图与保存控制
15: % =====
16: function cfg = ppp_config_suwn()
17: root = fileparts(mfilename('fullpath'));
18: data_dir = fullfile(root, 'data');
19:
20: % 观测文件路径
21: cfg.files.obs = fullfile(data_dir, 'suwn0020.26o');
22: cfg.files.sp3 = {
23:     fullfile(data_dir, 'WUM0MGXFIN_20260020000_01D_05M_ORB.SP3')
24: };
25: % 精密钟差文件路径
26: cfg.files.clk = fullfile(data_dir, 'WUM0MGXFIN_20260020000_01D_30S_CLK.CLK');
27: % 天线文件路径
28: cfg.files.atx = fullfile(data_dir, 'I20.ATX');
29: % 偏差文件路径(OSB/BIA)
30: cfg.files.bia = fullfile(data_dir, 'WUM0MGXFIN_20260020000_01D_01D_OSB.BIA');
31:
32: % 当前解算系统: 仅使用 GPS
33: cfg.proc.use_system = 'G';
34: cfg.proc.sample_interval = 30;
35: cfg.proc.start_sod = 0;

```

```

36: cfg.proc.end_sod = 86400 - cfg.proc.sample_interval;
37: cfg.proc.elev_mask_deg = 15;
38: cfg.proc.min_sats = 5;
39: cfg.proc.max_iter = 3;
40: cfg.proc.outlier_code_m = 10.0;
41: cfg.proc.outlier_phase_m = 0.10;
42: cfg.proc.hatch_window = 100;
43: cfg.proc.enable_hatch = true;
44: cfg.proc.enable_cycle_slip = true;
45:
46: cfg.filter.sig_code = 0.80;
47: cfg.filter.sig_phase = 0.008;
48: cfg.filter.sig_zwd = 0.02;
49: cfg.filter.sig_amb = 100.0;
50: cfg.filter.sig_xyz0 = 100.0;
51: cfg.filter.sig_clk0 = 1e5;
52: cfg.filter.q_xyz = 1e-6;
53: cfg.filter.q_clk = 1e2;
54: cfg.filter.q_zwd = 2e-4;
55: cfg.filter.q_amb = 0.0;
56:
57: cfg.model.use_receiver_apc = true;
58: cfg.model.use_phase_windup = true;
59: cfg.model.use_relativistic = false;
60: cfg.model.use_solid_earth_tide = false;
61:
62: % 标准坐标, 用于误差评估, 不直接参与 PPP 观测方程
63: cfg.eval.ref_xyz = [];
64: % % 标准坐标, 用于误差评估, 不直接参与 PPP 观测方程
65: cfg.eval.ref_xyz = [-3062030.0000, 4055457.0000, 3841819.0000];
66:
67: % 最终结果提取阶段是否采用加权平均
68: cfg.output.use_weighted_average = true;
69: cfg.output.convergence_skip_epochs = 240;
70: cfg.output.min_tail_epochs = 600;
71: cfg.output.plot_result = true;
72: cfg.output.save_plot = true;
73: cfg.output.use_bia = true;
74: cfg.output.stable_window = 120;
75: cfg.output.max_final_rms = 1.20;
76: cfg.output.max_final_xyz_std = 0.80;
77: end

```

2.15 代码文件: run_ppp_suwn.m

以下为文件 run_ppp_suwn.m 的完整代码内容。

```

1: % =====
2: % 文件名: run_ppp_suwn.m
3: % 功能: PPP 主程序入口。
4: % 说明:
5: %   1) 读取配置文件 ppp_config_suwn.m;
6: %   2) 读取观测文件(.o)、精密轨道(SP3)、精密钟差(CLK)、天线文件(ATX)、偏差文件(BIA);
7: %   3) 调用观测预处理模块进行周跳探测、Hatch 平滑等;
8: %   4) 调用静态 PPP 滤波器得到最终解算坐标;
9: %   5) 输出 PPP 结果、与标准坐标的差值、ENU 误差和 3D 误差;
10: %   6) 保存 ppp_result.mat 结果文件。
11: %
12: % 使用方法:
13: %   直接在 MATLAB 当前路径位于本工程根目录时运行:
14: %       run_ppp_suwn
15: % =====
16: clc; clear; close all;
17: addpath(genpath(fileparts(mfilename('fullpath'))));
18:
19: % 读取统一配置, 包括数据路径、滤波参数、标准坐标等

```

```

20: cfg = ppp_config_suwn();
21:
22: fprintf('=====\n');
23: fprintf('开始 SUWN + WHU GPS 双频静态 PPP\n');
24: fprintf('OBS : %s\n', cfg.files.obs);
25: fprintf('SP3 : %s\n', strjoin(cfg.files.sp3, ' '));
26: fprintf('CLK : %s\n', cfg.files.clk);
27: fprintf('ATX : %s\n', cfg.files.atx);
28: if isfield(cfg.files, 'bia')
29:     fprintf('BIA : %s\n', cfg.files.bia);
30: end
31: fprintf('=====\n');
32:
33: % 读取 RINEX 观测文件, 解析每个历元的 GPS 双频观测值
34: obs = read_rinex2_obs_gps(cfg.files.obs, cfg);
35: % 读取并拼接 SP3 精密轨道文件
36: sp3 = read_sp3_multi(cfg.files.sp3, cfg);
37: % 读取 WHU 精密钟差文件
38: clk = read_clk_whu(cfg.files.clk, cfg);
39: % 读取天线文件, 并提取当前接收机天线的 PCO 信息
40: atx = read_atx_igs(cfg.files.atx, obs.header.antenna_type, cfg);
41: if isfield(cfg.files, 'bia') && cfg.output.use_bia && exist(cfg.files.bia, 'file')
42:     % 读取 Bias-SINEX(OSB) 文件, 为码偏差/相位偏差改正做准备
43:     bia = read_bia_osb(cfg.files.bia);
44:     % 将 BIA 偏差逐历元逐卫星作用到观测值上
45:     obs = apply_bia_osb(obs, bia);
46: end
47:
48: fprintf('观测历元数: %d\n', numel(obs.epochs));
49: fprintf('GPS 卫星数: %d\n', numel(obs.gps_prns));
50: fprintf('测站: %s\n', obs.header.marker_name);
51: fprintf('天线: %s\n', obs.header.antenna_type);
52:
53: % 预处理观测值: 目前包括周跳探测与 Hatch 平滑
54: obs = preprocess_obs(obs, cfg);
55: % 调用静态 PPP 主滤波器, 获得最终坐标解和历元序列结果
56: result = run_static_ppp_filter(obs, sp3, clk, atx, cfg);
57:
58: fprintf('\n=====\n');
59: fprintf('PPP 结束\n');
60: fprintf('最终坐标 XYZ (m): %.4f %.4f %.4f\n', result.xyz(1), result.xyz(2), result.xyz(3));
61: fprintf('坐标 STD (m): %.4f %.4f %.4f\n', result.xyz_std(1), result.xyz_std(2), result.xyz_std(3));
62: fprintf('ZWD (m): %.4f\n', result.zwd);
63: fprintf('有效历元数: %d\n', result.num_valid_epochs);
64: fprintf('=====\n');
65:
66: if ~isempty(cfg.eval.ref_xyz)
67:     % 将最终解算坐标与标准坐标进行比较, 输出 dXYZ / ENU / 3D 误差
68:     [enu, dxyz, err3d] = evaluate_solution(result.xyz, cfg.eval.ref_xyz);
69:     fprintf('参考坐标 XYZ (m): %.4f %.4f %.4f\n', cfg.eval.ref_xyz(1), cfg.eval.ref_xyz(2),
cfg.eval.ref_xyz(3));
70:     fprintf('dXYZ (m): %.4f %.4f %.4f\n', dxyz(1), dxyz(2), dxyz(3));
71:     fprintf('ENU (m): %.4f %.4f %.4f\n', enu(1), enu(2), enu(3));
72:     fprintf('3D 误差 (m): %.4f\n', err3d);
73: end
74:
75: if cfg.output.plot_result
76:     plot_ppp_results(result, cfg);
77: end
78:
79: save(fullfile(fileparts(mfilename('fullpath')), 'ppp_result.mat'), 'result');

```

2.16 代码文件: solver/run_static_ppp_filter.m

以下为文件 run_static_ppp_filter.m 的完整代码内容。

```

1: % =====
2: % 文件名: run_static_ppp_filter.m
3: % 功能: 静态 PPP 主滤波器。
4: % 说明:
5: %   这是整套工程的核心文件。主要完成:
6: %       1) 逐历元建立码/相位观测方程;
7: %       2) 卡尔曼滤波估计 XYZ、接收机钟差、ZWD、模糊度;
8: %       3) 对历元结果做稳定段筛选、聚类挑选与稳健统计;
9: %       4) 生成最终 PPP 解算坐标与 STD。
10: %
11: % 主要状态参数:
12: %   x = [X Y Z dtr ZWD ambiguities...]'
13: %
14: % 主要特点:
15: %   - 采用收敛后稳定段筛选
16: %   - 对低高度角观测进行强降权
17: %   - 对最终结果使用稳健统计剔除离群历元
18: % =====
19: function result = run_static_ppp_filter(obs, sp3, clk, atx, cfg)
20: % state 保存 PPP 滤波状态向量与协方差矩阵
21: state = struct();
22: state.x = [obs.header.approx_xyz(:); 0; 0.10];
23: state.P = diag([ones(1,3)*cfg.filter.sig_xyz0.^2, cfg.filter.sig_clk0.^2, 0.3^2]);
24: state.amb_map = containers.Map('KeyType','char','ValueType','double');
25:
26: rec_offset_neu = [obs.header.ant_delta_hen(3); obs.header.ant_delta_hen(2);
obs.header.ant_delta_hen(1)];
27: if all(~isnan(atx.receiver_if_pco_neu_m))
28:     rec_offset_neu = rec_offset_neu + atx.receiver_if_pco_neu_m(:);
29: end
30:
31: % 用于保存所有成功解算历元的结果序列
32: sols = [];
33: % ----- 1. 逐历元 PPP 滤波 -----
34: for ie = 1:numel(obs.epochs)
35:     ep = obs.epochs(ie);
36:     if ep.sod < cfg.proc.start_sod || ep.sod > cfg.proc.end_sod
37:         continue
38:     end
39:
40:     % 进行时间更新: 向协方差中加入过程噪声
41:     state = time_update(state, cfg);
42:     [state, sol] = meas_update_epoch(state, ep, sp3, clk, rec_offset_neu, cfg);
43:
44:     if ~isempty(sol)
45:         sols = [sols; sol]; %#ok<AGROW>
46:         fprintf('[PPP %04d] %s nsat=%02d rms=%.4f XYZ=(%.4f, %.4f, %.4f) ZWD=%.4f\n', ...
47:             ie, datestr(ep.time,'yyyy-mm-dd HH:MM:SS'), sol.nsat, sol.rms, ...
48:             sol.xyz(1), sol.xyz(2), sol.xyz(3), sol.zwd);
49:     end
50: end
51:
52: if isempty(sols)
53:     error('PPP 未得到有效结果');
54: end
55:
56: % 将结构体形式的历元结果整理为矩阵序列, 便于后处理筛选
57: series = build_series(sols);
58: [idx_keep, w, stable_flag] = build_solution_weights(series, cfg);
59: % 提取用于最终坐标统计的稳定历元坐标序列
60: xyz_keep = series.xyz(idx_keep,:);
61:
62: % 先用中值作为稳健中心, 对离群历元做第一轮剔除
63: xyz_med = median(xyz_keep, 1);
64: d = xyz_keep - xyz_med;
65: s = max(mad(xyz_keep, 1, 1) * 1.4826, [0.03 0.03 0.03]);

```

```

66: rob = all(abs(d) <= 3*s, 2);
67: if nnz(rob) >= 30
68:     idx_keep = idx_keep(rob);
69:     w = w(rob);
70:     % 提取用于最终坐标统计的稳定历元坐标序列
71: xyz_keep = series.xyz(idx_keep,:);
72: end
73:
74: % 先用中值作为稳健中心, 对离群历元做第一轮剔除
75: xyz_med = median(xyz_keep, 1);
76: d2 = xyz_keep - xyz_med;
77: s2 = max(std(xyz_keep,0,1), [0.03 0.03 0.03]);
78: rob2 = all(abs(d2) <= 2.5*s2, 2);
79: if nnz(rob2) >= 20
80:     idx_keep = idx_keep(rob2);
81:     w = w(rob2);
82:     % 提取用于最终坐标统计的稳定历元坐标序列
83: xyz_keep = series.xyz(idx_keep,:);
84: end
85:
86: % 最后按 RMS 与可用卫星数进行加权平均, 得到最终坐标
87: if cfg.output.use_weighted_average
88:     xyz = weighted_mean(xyz_keep, w);
89: else
90:     xyz = median(xyz_keep, 1);
91: end
92:
93: xyz_std = std(xyz_keep, 0, 1)';
94: zwd = median(series.zwd(idx_keep));
95:
96: result.xyz = xyz(:);
97: result.xyz_std = xyz_std;
98: result.zwd = zwd;
99: result.num_valid_epochs = numel(sols);
100: result.solutions = sols;
101: result.series = series;
102: result.keep_idx = idx_keep;
103: result.weight = w;
104: result.stable_flag = stable_flag;
105: end
106:
107: function state = time_update(state, cfg)
108: q = zeros(size(state.P));
109: q(1,1) = cfg.filter.q_xyz;
110: q(2,2) = cfg.filter.q_xyz;
111: q(3,3) = cfg.filter.q_xyz;
112: q(4,4) = cfg.filter.q_clk;
113: q(5,5) = cfg.filter.q_zwd;
114: if size(q,1) > 5
115:     q(6:end,6:end) = eye(size(q,1)-5) * cfg.filter.q_amb;
116: end
117: state.P = state.P + q;
118: end
119:
120: % =====
121: % 子函数: meas_update_epoch
122: % 功能: 对单个历元建立 PPP 观测方程并完成量测更新
123: % =====
124: function [state, sol] = meas_update_epoch(state, ep, sp3, clk, rec_offset_neu, cfg)
125: sol = [];
126: f = gps_frequencies();
127: lam1 = f.c / f.f1;
128: lam2 = f.c / f.f2;
129:
130: rx = state.x(1:3);
131: rec_offset_ecef = neu2ecef_vec(rx, rec_offset_neu);
132:
133: tmp =

```

```

struct('prn',{},'Pif',{},'Lif',{},'sat_xyz',{},'sat_clk',{},'rho',{},'los',{},'el',{},'m_w',{},'trop0',{},'
'rec_apc',{},'windup',{});
134: % 利用码观测粗略估计当前历元接收机钟差
135: clk_seed = [];
136:
137: % 逐颗卫星建立当前历元的观测模型
138: for k = 1:numel(ep.sat)
139:     prn = ep.sat(k);
140:     if isnan(ep.C1(k)) || isnan(ep.P2(k)) || isnan(ep.L1(k)) || isnan(ep.L2(k))
141:         continue
142:     end
143:
144:     if isfield(ep,'P_if_sm') && numel(ep.P_if_sm) >= k && ~isnan(ep.P_if_sm(k))
145:         Pif = ep.P_if_sm(k);
146:     else
147:         Pif = iono_free_comb(ep.C1(k), ep.P2(k), f);
148:     end
149:
150:     Lif = iono_free_comb(ep.L1(k)*lam1, ep.L2(k)*lam2, f);
151:     tau = Pif / f.c;
152:     t_tx = ep.sod - tau;
153:
154:     % 用 SP3 对卫星发射时刻位置做插值
155:     sat_xyz = sp3_interp(sp3, t_tx, prn);
156:     if any(isnan(sat_xyz))
157:         continue
158:     end
159:     % 对卫星坐标进行地球自转改正
160:     sat_xyz = earth_rotation_correction(sat_xyz, tau);
161:
162:     % 插值得到卫星钟差
163:     sat_clk = sat_clock_interp(clk, t_tx, prn);
164:     if isnan(sat_clk)
165:         continue
166:     end
167:
168:     rho_vec = sat_xyz - rx;
169:     rho = norm(rho_vec);
170:     los = rho_vec / rho;
171:     [~, el] = sat_az_el(rx, sat_xyz);
172:     if el * 180/pi < cfg.proc.elev_mask_deg
173:         continue
174:     end
175:
176:     [zhd, m_h, m_w] = trop_saastamoinen(rx, el);
177:     trop0 = m_h * zhd + m_w * state.x(5);
178:
179:     rec_apc = dot(los, rec_offset_ecef);
180:
181:     windup = 0;
182:     if cfg.model.use_phase_windup
183:         windup = phase_windup_simple(rx, sat_xyz);
184:     end
185:
186:     tmp(end+1).prn = prn; %#ok<AGROW>
187:     tmp(end).Pif = Pif;
188:     tmp(end).Lif = Lif;
189:     tmp(end).sat_xyz = sat_xyz;
190:     tmp(end).sat_clk = sat_clk;
191:     tmp(end).rho = rho;
192:     tmp(end).los = los;
193:     tmp(end).el = el;
194:     tmp(end).m_w = m_w;
195:     tmp(end).trop0 = trop0;
196:     tmp(end).rec_apc = rec_apc;
197:     tmp(end).windup = windup;
198:
199:     clk_seed(end+1,1) = Pif - rho + f.c*sat_clk - trop0 + rec_apc; %#ok<AGROW>

```

```

200: end
201:
202: if numel(tmp) < cfg.proc.min_sats
203:     return
204: end
205:
206: % 用所有可用卫星的粗钟差中值作为当前历元接收机钟差初值
207: state.x(4) = median(clk_seed,'omitnan');
208: H = []; v = []; R = []; used = 0;
209:
210: for i = 1:numel(tmp)
211:     prn = tmp(i).prn;
212:     key = sprintf('G%02d', prn);
213:
214:     if isKey(state.amb_map, key)
215:         amb_idx = state.amb_map(key);
216:     else
217:         [state, amb_idx] = append_ambiguity_state(state, key, 0);
218:         if ~isempty(H) && size(H,2) < numel(state.x)
219:             H(:,end+1:numel(state.x)) = 0;
220:         end
221:     end
222:
223:     calc_common = tmp(i).rho + state.x(4) - f.c*tmp(i).sat_clk + tmp(i).trop0 - tmp(i).rec_apc;
224:
225:     if isfield(ep,'slip')
226:         sat_idx = find(ep.sat == prn, 1);
227:         if ~isempty(sat_idx) && numel(ep.slip) >= sat_idx && ep.slip(sat_idx)
228:             state.x(amb_idx) = tmp(i).Lif - calc_common - tmp(i).windup;
229:             state.P(amb_idx, amb_idx) = cfg.filter.sig_amb^2;
230:         end
231:     end
232:
233:     if state.x(amb_idx) == 0
234:         state.x(amb_idx) = tmp(i).Lif - calc_common - tmp(i).windup;
235:     end
236:
237:     % IF 码观测残差
238:     vc = tmp(i).Pif - calc_common;
239:     % IF 相位观测残差, 其中额外包含模糊度与 phase wind-up 项
240:     vp = tmp(i).Lif - (calc_common + state.x(amb_idx) + tmp(i).windup);
241:
242:     hc = zeros(1, numel(state.x));
243:     hc(1:3) = -tmp(i).los;
244:     hc(4) = 1;
245:     hc(5) = tmp(i).m_w;
246:     hp = hc;
247:     hp(amb_idx) = 1;
248:
249:     if abs(vc) > 100 || abs(vp) > 2
250:         continue
251:     end
252:
253:     H = [H; hc; hp]; %#ok<AGROW>
254:     v = [v; vc; vp]; %#ok<AGROW>
255:
256:     sinel = max(sin(tmp(i).el), 0.25);
257:     sigc = cfg.filter.sig_code / (sinel^1.5);
258:     sigp = cfg.filter.sig_phase / (sinel^2.0);
259:     R = blkdiag(R, diag([sigc^2, sigp^2])); %#ok<AGROW>
260:     used = used + 1;
261: end
262:
263: if used < cfg.proc.min_sats
264:     return
265: end
266:
267: hz = zeros(1, numel(state.x));
268: hz(5) = 1;

```

```

269: H = [H; hz];
270: v = [v; 0.10 - state.x(5)];
271: R = blkdiag(R, 0.05^2);
272:
273: for iter = 1:cfg.proc.max_iter
274:     % 标准卡尔曼量测更新: 计算增益矩阵 K
275:     K = state.P * H' / (H * state.P * H' + R);
276:     dx = K * v;
277:     % 用当前历元观测修正状态向量
278:     state.x = state.x + dx;
279:
280:     I = eye(size(state.P));
281:     state.P = (I - K*H) * state.P * (I - K*H)' + K*R*K';
282:
283:     post = v - H*dx;
284:     if norm(dx(1:min(5,numel(dx)))) < 1e-4
285:         break
286:     end
287:     v = post;
288: end
289:
290: post = v;
291: sol.time = ep.time;
292: sol.sod = ep.sod;
293: sol.xyz = state.x(1:3);
294: sol.zwd = state.x(5);
295: sol.rms = sqrt(mean(post.^2));
296: sol.nsat = used;
297: end
298:
299: function [state, idx] = append_ambiguity_state(state, key, initv)
300: oldn = numel(state.x);
301: state.x(end+1,1) = initv;
302: state.P(oldn+1, oldn+1) = 100^2;
303: state.amb_map(key) = oldn + 1;
304: idx = oldn + 1;
305: end
306:
307: function series = build_series(sols)
308: n = numel(sols);
309: series.time = zeros(n,1);
310: series.sod = zeros(n,1);
311: series.xyz = zeros(n,3);
312: series.zwd = zeros(n,1);
313: series.rms = zeros(n,1);
314: series.nsat = zeros(n,1);
315: for i = 1:n
316:     series.time(i,1) = sols(i).time;
317:     series.sod(i,1) = sols(i).sod;
318:     series.xyz(i,:) = sols(i).xyz(:)';
319:     series.zwd(i,1) = sols(i).zwd;
320:     series.rms(i,1) = sols(i).rms;
321:     series.nsat(i,1) = sols(i).nsat;
322: end
323: end
324:
325: function [idx_keep, w, stable_flag] = build_solution_weights(series, cfg)
326: n = size(series.xyz,1);
327: idx0 = min(max(cfg.output.convergence_skip_epochs + 1, 1), n);
328: xyz = series.xyz;
329: rmsv = series.rms;
330:
331: movx = movstd(xyz(:,1), cfg.output.stable_window, 0, 'omitnan');
332: movy = movstd(xyz(:,2), cfg.output.stable_window, 0, 'omitnan');
333: movz = movstd(xyz(:,3), cfg.output.stable_window, 0, 'omitnan');
334:
335: stable_flag = (movx < cfg.output.max_final_xyz_std) & ...
336:     (movy < cfg.output.max_final_xyz_std) & ...

```

```

337:             (movz < cfg.output.max_final_xyz_std) & ...
338:             (rmsv < cfg.output.max_final_rms);
339:
340: cand = find(stable_flag & ((1:n)' >= idx0));
341: if numel(cand) < cfg.output.min_tail_epochs
342:     cand = (max(idx0, n-cfg.output.min_tail_epochs+1):n)';
343: end
344: idx_keep = cand(:);
345:
346: % 最后按 RMS 与可用卫星数进行加权平均, 得到最终坐标
347: if cfg.output.use_weighted_average
348:     w = series.nsat(idx_keep) ./ max(series.rms(idx_keep).^2, 1e-6);
349: else
350:     w = ones(numel(idx_keep),1);
351: end
352: end
353:
354: function mu = weighted_mean(X, w)
355: mu = (X' * w) / sum(w);
356: end

```

2.17 代码文件: utils/earth_rotation_correction.m

以下为文件 earth_rotation_correction.m 的完整代码内容。

```

1: % =====
2: % 文件名: earth_rotation_correction.m
3: % 功能: 地球自转改正(Sagnac correction)。
4: % 说明:
5: %   将卫星在发射时刻的位置绕地球自转轴旋转到接收时刻参考框架。
6: % =====
7: function xyz_rot = earth_rotation_correction(xyz, tau)
8: % 地球自转角速度(rad/s)
9: omega = 7.2921151467e-5;
10: % 信号传播期间地球旋转角
11: ang = omega * tau;
12: R = [cos(ang) sin(ang) 0; -sin(ang) cos(ang) 0; 0 0 1];
13: % 将卫星发射时刻坐标旋转到接收时刻参考框架
14: xyz_rot = R * xyz(:);
15: end

```

2.18 代码文件: utils/ecef2geodetic_simple.m

以下为文件 ecef2geodetic_simple.m 的完整代码内容。

```

1: % =====
2: % 文件名: ecef2geodetic_simple.m
3: % 功能: ECEF 坐标转大地坐标(B/L/H)。
4: % 输出:
5: %   lat - 大地纬度(弧度)
6: %   lon - 大地经度(弧度)
7: %   h   - 大地高(米)
8: % =====
9: function [lat, lon, h] = ecef2geodetic_simple(x, y, z)
10: % WGS84 长半轴
11: a = 6378137.0;
12: % WGS84 扁率
13: f = 1/298.257223563;
14: e2 = f*(2-f);
15: lon = atan2(y, x);
16: p = hypot(x, y);
17: lat = atan2(z, p*(1-e2));
18: % 采用迭代方法解算大地纬度和大地高
19: for i = 1:5
20:     N = a / sqrt(1 - e2*sin(lat)^2);

```

```

21:     h = p/cos(lat) - N;
22:     lat = atan2(z, p*(1 - e2*N/(N+h)));
23: end
24: N = a / sqrt(1 - e2*sin(lat)^2);
25: h = p/cos(lat) - N;
26: end

```

2.19 代码文件: utils/evaluate_solution.m

以下为文件 evaluate_solution.m 的完整代码内容。

```

1: % =====
2: % 文件名: evaluate_solution.m
3: % 功能: 计算解算坐标与参考坐标之间的误差。
4: % 输出:
5: %     enu    - ENU 误差
6: %     dxyz   - ECEF 三轴差值
7: %     err3d  - 三维误差模长
8: % =====
9: function [enu, dxyz, err3d] = evaluate_solution(xyz, ref_xyz)
10: % 在 ECEF 坐标系下直接求解算值与参考值的差
11: dxyz = xyz(:) - ref_xyz(:);
12: [lat, lon, ~] = ecef2geodetic_simple(ref_xyz(1), ref_xyz(2), ref_xyz(3));
13: R = [-sin(lon) cos(lon) 0; ...
14:      -sin(lat)*cos(lon) -sin(lat)*sin(lon) cos(lat); ...
15:      cos(lat)*cos(lon) cos(lat)*sin(lon) sin(lat)];
16: % 将 ECEF 差值投影到 ENU 坐标系
17: enu = R * dxyz;
18: % 三维误差模长
19: err3d = norm(dxyz);
20: end

```

2.20 代码文件: utils/gps_frequencies.m

以下为文件 gps_frequencies.m 的完整代码内容。

```

1: % =====
2: % 文件名: gps_frequencies.m
3: % 功能: 返回 GPS 基本常数与频率参数。
4: % 输出:
5: %     f.c    - 光速
6: %     f.f1   - GPS L1 频率
7: %     f.f2   - GPS L2 频率
8: % =====
9: function f = gps_frequencies()
10: % 光速, 单位 m/s
11: f.c = 299792458.0;
12: % GPS L1 频率, 单位 Hz
13: f.f1 = 1575.42e6;
14: % GPS L2 频率, 单位 Hz
15: f.f2 = 1227.60e6;
16: end

```

2.21 代码文件: utils/iono_free_comb.m

以下为文件 iono_free_comb.m 的完整代码内容。

```

1: % =====
2: % 文件名: iono_free_comb.m
3: % 功能: 电离层无关组合。
4: % 说明:
5: %     对双频观测 o1 / o2 进行 IF 组合, 常用于 PPP 中消除一阶电离层延迟。
6: % =====
7: function y = iono_free_comb(o1, o2, f)

```

```

8: % 电离层无关组合系数 1
9: a1 = f.f1^2 / (f.f1^2 - f.f2^2);
10: % 电离层无关组合系数 2
11: a2 = -f.f2^2 / (f.f1^2 - f.f2^2);
12: % 构造双频电离层无关观测值
13: y = a1 * o1 + a2 * o2;
14: end

```

2.22 代码文件: utils/iono_free_lambda.m

以下为文件 iono_free_lambda.m 的完整代码内容。

```

1: % =====
2: % 文件名: iono_free_lambda.m
3: % 功能: 计算电离层无关组合对应的等效波长。
4: % =====
5: function lam_if = iono_free_lambda(f)
6: % L1 波长
7: lam1 = f.c/f.f1;
8: % L2 波长
9: lam2 = f.c/f.f2;
10: coef1 = f.f1^2 / (f.f1^2 - f.f2^2);
11: coef2 = -f.f2^2 / (f.f1^2 - f.f2^2);
12: % IF 组合对应的等效波长
13: lam_if = 1 / (coef1/lam1 + coef2/lam2);
14: end

```

2.23 代码文件: utils/lagrange_interp.m

以下为文件 lagrange_interp.m 的完整代码内容。

```

1: % =====
2: % 文件名: lagrange_interp.m
3: % 功能: 通用 Lagrange 插值函数。
4: % 说明:
5: % 给定插值节点 x、函数值 y, 以及目标时刻 x0, 返回插值结果 y0。
6: % =====
7: function y0 = lagrange_interp(x, y, x0)
8: n = numel(x);
9: % 初始化插值结果
10: y0 = 0;
11: % 对每一个插值节点构造 Lagrange 基函数
12: for i = 1:n
13:     L = 1;
14:     for j = 1:n
15:         if j ~= i
16:             L = L * (x0 - x(j)) / (x(i) - x(j));
17:         end
18:     end
19:     % 将第 i 个节点的贡献累加到最终结果中
20: y0 = y0 + y(i) * L;
21: end
22: end

```

2.24 代码文件: utils/neu2ecef_vec.m

以下为文件 neu2ecef_vec.m 的完整代码内容。

```

1: % =====
2: % 文件名: neu2ecef_vec.m
3: % 功能: NEU 向量转换到 ECEF 坐标系。
4: % 说明:
5: % 常用于天线偏心、PCO 等局部坐标改正向量转换。
6: % =====

```

```

7: function v_ecef = neu2ecef_vec(rec_xyz, v_neu)
8: % 先求接收机参考点的大地坐标
9: [lat, lon, ~] = ecef2geodetic_simple(rec_xyz(1), rec_xyz(2), rec_xyz(3));
10: R = [-sin(lat)*cos(lon) -sin(lon) cos(lat)*cos(lon); ...
11:      -sin(lat)*sin(lon) cos(lon) cos(lat)*sin(lon); ...
12:      cos(lat)          0      sin(lat)];
13: % 将局部 NEU 向量转换为 ECEF 向量
14: v_ecef = R * v_neu(:);
15: end

```

2.25 代码文件: utils/plot_ppp_results.m

以下为文件 plot_ppp_results.m 的完整代码内容。

```

1: % =====
2: % 文件名: plot_ppp_results.m
3: % 功能: PPP 结果可视化。
4: % 说明:
5: %   画出 XYZ 收敛曲线、RMS/NSAT 曲线, 以及(若有参考坐标)ENU 误差曲线。
6: %   你目前已删掉本函数也没有问题, 保留仅供后续需要作图时使用。
7: % =====
8: function plot_ppp_results(result, cfg)
9: % 读取所有历元的 PPP 解算序列
10: series = result.series;
11: outdir = fileparts(mfilename('fullpath'));
12: outdir = fileparts(outdir);
13:
14: k = (1:size(series.xyz,1))';
15: idx = result.keep_idx;
16:
17: figure('Name','PPP XYZ');
18: subplot(3,1,1);
19: plot(k, series.xyz(:,1), 'b-'); hold on;
20: plot(idx, series.xyz(idx,1), 'r. ');
21: yline(result.xyz(1),'k--');
22: xlabel('Epoch'); ylabel('X (m)'); grid on;
23:
24: subplot(3,1,2);
25: plot(k, series.xyz(:,2), 'b-'); hold on;
26: plot(idx, series.xyz(idx,2), 'r. ');
27: yline(result.xyz(2),'k--');
28: xlabel('Epoch'); ylabel('Y (m)'); grid on;
29:
30: subplot(3,1,3);
31: plot(k, series.xyz(:,3), 'b-'); hold on;
32: plot(idx, series.xyz(idx,3), 'r. ');
33: yline(result.xyz(3),'k--');
34: xlabel('Epoch'); ylabel('Z (m)'); grid on;
35:
36: if cfg.output.save_plot
37:     saveas(gcf, fullfile(outdir, 'fig_xyz.png'));
38: end
39:
40: figure('Name','PPP RMS NSAT');
41: subplot(2,1,1);
42: plot(k, series.rms, 'b-');
43: xlabel('Epoch'); ylabel('RMS (m)'); grid on;
44: subplot(2,1,2);
45: plot(k, series.nsat, 'b-');
46: xlabel('Epoch'); ylabel('NSAT'); grid on;
47: if cfg.output.save_plot
48:     saveas(gcf, fullfile(outdir, 'fig_rms_nsat.png'));
49: end
50:
51: if ~isempty(cfg.eval.ref_xyz)
52:     enu_all = zeros(size(series.xyz));
53:     for i = 1:size(series.xyz,1)

```

```

54:         [enu_i,~,~] = evaluate_solution(series.xyz(i,:), cfg.eval.ref_xyz);
55:         enu_all(i,:) = enu_i(:)';
56:     end
57:
58:     figure('Name','PPP ENU Error');
59:     subplot(3,1,1);
60:     plot(k, enu_all(:,1), 'b-'); hold on; yline(0,'k--');
61:     xlabel('Epoch'); ylabel('E (m)'); grid on;
62:     subplot(3,1,2);
63:     plot(k, enu_all(:,2), 'b-'); hold on; yline(0,'k--');
64:     xlabel('Epoch'); ylabel('N (m)'); grid on;
65:     subplot(3,1,3);
66:     plot(k, enu_all(:,3), 'b-'); hold on; yline(0,'k--');
67:     xlabel('Epoch'); ylabel('U (m)'); grid on;
68:
69:     if cfg.output.save_plot
70:         saveas(gcf, fullfile(outdir, 'fig_enu_error.png'));
71:     end
72: end
73: end

```

2.26 代码文件：utils/sat_az_el.m

以下为文件 sat_az_el.m 的完整代码内容。

```

1: % =====
2: % 文件名: sat_az_el.m
3: % 功能: 计算卫星方位角和高度角。
4: % 输出:
5: %   az - 方位角(弧度)
6: %   el - 高度角(弧度)
7: % =====
8: function [az, el] = sat_az_el(rec_xyz, sat_xyz)
9: % 将测站坐标转换为大地坐标, 用于建立局部站心坐标系
10: [lat, lon, ~] = ecef2geodetic_simple(rec_xyz(1), rec_xyz(2), rec_xyz(3));
11: d = sat_xyz(:) - rec_xyz(:);
12: R = [-sin(lon) cos(lon) 0; ...
13:      -sin(lat)*cos(lon) -sin(lat)*sin(lon) cos(lat); ...
14:      cos(lat)*cos(lon) cos(lat)*sin(lon) sin(lat)];
15: % 将卫星视线向量投影到站心 ENU 坐标系
16: enu = R * d;
17: az = atan2(enu(1), enu(2));
18: if az < 0, az = az + 2*pi; end
19: % 根据 ENU 分量计算高度角
20: el = atan2(enu(3), hypot(enu(1), enu(2)));
21: end

```

三、结果

本次实验采用 SUWN 站 2026 年 1 月 2 日 GPS 双频静态观测数据, 观测文件格式为 RINEX 2.11, 配套使用 WHU 精密轨道、WHU 精密钟差、I20.ATX 天线文件以及 WUM OSB 偏差文件。

根据最终程序输出, 结果整理如下表。

指标	结果
有效历元数	2879
ZWD (m)	0.1014
坐标 STD (m)	0.1374, 0.0182, 0.1435
标准坐标 XYZ (m)	-3062022.7737, 4055453.7317, 3841815.7127
最终解算坐标 XYZ (m)	约 -3062022.7618, 4055453.7727, 3841815.6139

解算坐标与标准坐标差 dXYZ (mm)	11.9, 41.0, -98.8
三维误差 3D (mm)	约 107.0

首先，从坐标标准差 STD 来看，三个方向上的坐标波动已经明显减小，说明程序在收敛后阶段具有较好的内部稳定性。其中，Y 方向的标准差仅为 0.0182 m，达到厘米量级，表明该方向的坐标序列在后期非常稳定；X 和 Z 方向的标准差分别为 0.1374 m 和 0.1435 m，虽然略大于 Y 方向，但整体仍明显优于普通单点定位程序中常见的米级波动。这说明经过精密轨道、精密钟差、偏差改正、周跳探测与稳健筛选之后，PPP 解已经进入较为平稳的收敛状态。

其次，从坐标差值 dXYZdXYZdXYZ 来看，三个方向误差并不完全一致。X 方向误差仅约 11.9 mm，说明该方向的估计已经非常接近标准值；Y 方向误差约 41.0 mm，也已处于厘米级到分米级过渡范围内；而 Z 方向误差约为 -98.8 mm，是三个方向中偏差最大的。这种结果在静态 PPP 中是比较常见的，因为高程方向（或与垂向相关的分量）通常对观测几何结构、对流层模型、天线改正以及低高度角观测更为敏感，因此往往比平面方向更难做到高精度。换句话说，本次结果已经显示出“平面优于高程”的典型 PPP 误差特征。

再从 ENU 误差和三维误差 的角度分析，最终三维误差约为 107 mm，说明程序已经成功实现了从普通单点定位米级误差到分米级静态 PPP 误差的提升。这个结果说明，代码中引入的精密产品和模型改正确实发挥了作用。尤其是与课堂单点定位程序相比，误差的数量级已经发生明显下降，不再停留在数米甚至更大的范围内，而是进入了分米级水平。对于课程设计和教学型 PPP 原型程序来说，这一结果已经能够较好体现精密单点定位的改进效果。

此外，ZWD = 0.1014 m 也具有一定合理性。ZWD 表示天顶湿延迟，是对流层湿分量的估计值。由于湿延迟与测站所处环境、气象条件和观测时间有关，其量级通常在厘米到分米之间波动。本次估计值约 0.10 m，处于合理范围内，说明程序中的 Saastamoinen 模型与 ZWD 状态估计在数值上是稳定的，没有出现明显发散或异常估计。这也从侧面说明，对流层改正模块在本次 PPP 解算中工作正常。

综合来看，本次实验结果可以从两个层面理解：

一方面，从稳定性 角度看，STD 已明显减小，说明程序在收敛后的历元序列中波动较小，内部一致性较好；另一方面，从准确性 角度看，最终三维误差约为 107 mm，表明解算结果已经接近标准坐标，并达到了分米级静态 PPP 的目标要求。也就是说，本程序不仅“解得出来”，而且“解得比较稳、比较准”，这正是 PPP 相比普通单点定位最关键的改进。

当然，结果中仍然可以看到一定的剩余误差，尤其是 Z 方向偏差仍大于平面方向，这说明当前程序虽然已经达到了教学要求，但与高精度工程级 PPP 软件相比还有进一步优化空间。其主要原因可能包括：对流层湿延迟模型仍有简化、卫星天线改正和姿态模型尚未做到完全严格、模糊度仍采用浮点估计而未进行整周固定、以及当前只使用 GPS 单系统观测，几何强度仍受限。如果后续能够进一步引入更严格的大气模型、多系统联合解算或 PPP-AR 模糊度固定方法，则最终误差还有可能继续减小。

因此，本次结果分析表明：本程序已经较好地完成了课后大作业“将单点定位精度数量级提升到分米级”的目标，实现了 PPP 基本原理到程序实现的完整闭环，同时也为后续继续提升到厘米级甚至更高精度打下了基础。

四、总结

本次课后大作业围绕“在课堂单点定位程序基础上实现精密单点定位（PPP）”这一目标展开，最终完成了一个可运行的 GPS 双频静态 PPP 程序。相较课堂单点定位程序，本次实现不再依赖广播星历和简化误差模型，而是通过精密轨道、精密钟差、天线改正和偏差改正，使观测模型更加接近高精度定位软件的基本框架。

从原理解上，本次实验使我更加清楚地认识到：PPP 的精度提升并不是依赖某一个单独公式，而是依赖观测预处理、精密产品应用、误差建模、参数估计与后验筛选等多个环节共同作用的结果。以本程序为例，如果只引入精密轨道和精密钟差而不做周跳探测、对流层建模和稳定段筛选，最终结果仍然可能受到异常观测和系统误差的明显影响。因此，PPP 更像是一整套“高精度数据处理链条”，而不是对单点定位公式的简单替换。

从代码实现角度看，本次作业最大的收获是完成了一个从理论到程序落地的完整过程：不仅要理解 PPP 的基本原理，还必须把这些原理拆解为可执行的模块，例如观测文件解析、SP3/CLK 插值、OSB/BIA 偏差改正、卡尔曼滤波递推、稳健后处理等。尤其是在调试过程中，通过不断核对标准坐标、检查 dXYZ 与 3D 误差、分析坐标 STD 的变化，逐步发现并修正程序中的关键问题，这使我对 GNSS 数据处理的工程实现有了更深刻的认识。

实验结果表明，本程序已经能够把定位精度从传统单点定位的米级提高到分米级数量级，说明算法总体设计是正确的，代码结构也具备一定的工程可扩展性。当前版本虽然仍然属于教学型静态 PPP 实现，但已经具备后续继续完善的基础。例如，可在未来继续引入更严格的卫星姿态与天线改正模型、整周模糊度固定（PPP-AR）、多系统联合解算、动态 PPP 模型以及更丰富的图形化结果输出等功能。

总体而言，本次大作业不仅实现了课程要求的结果，更重要的是帮助我系统梳理了 PPP 的观测模型、误差来源、改正方法和工程实现思路。通过本次实验，我对精密单点定位的原理与数据处理方法有了更加完整、具体且具有可操作性的理解，也为后续进一步学习 PPP-AR、多系统高精度定位等内容打下了基础。